Documento Técnico: Arquitectura e Implementación de la Aplicación

1. Introducción

Este documento técnico describe la arquitectura e implementación inicial de la aplicación solicitada para el ejercicio 4 de la entrega de Sistemas Operativos.

2. Descripción de la arquitectura y sus componentes

La aplicación cuenta con tres capas:

- Capa de entrada: Es a la que apunta el usuario a la hora de entrar al sistema, genera un punto único de ingreso y distribuye el tráfico de la red.
- Capa de aplicación: Es dónde se encuentra la aplicación, para el ejemplo usamos un escalado horizontal de tres instancias, estas son todas idénticas entre sí.
- Capa de datos: Es donde se encuentra almacenada la base de datos, tiene persistencia y es compartida por las tres instancias de la aplicación.

2.1 Capa de entrada

La primera capa se encuentra compuesta por un balanceador Nginx que actúa como punto único de partida. Este se encarga de recibir todas las peticiones http provenientes de los clientes y las distribuye entre las múltiples instancias.
Esto es realizado mediante el algoritmo de Round Robin (método por defecto), permitiendo una distribución equitativa entre todas los servidores de backend.

2.2 Capa de aplicación

La segunda capa está compuesta por múltiples instancias idénticas de la aplicación web. Por defecto, se ejecutan tres réplicas del mismo contenedor, cada una escuchando en el puerto 5000.
Estas sesiones carecen de estado, por lo cual no persisten ningún tipo de información.

2.3 Capa de persistencia

La tercera y última capa está compuesta por una instancia que contiene Redis que funciona como base de datos en memoria compartida.
Todas las instancias de la capa de aplicación se conectan a esta instancia, garantizando la consistencia de los datos.

Esta instancia utiliza un volumen de docker, permitiendo persistir su información y evitar que en reinicios/apagados se pueda perder.

3. Explicación de las decisiones técnicas

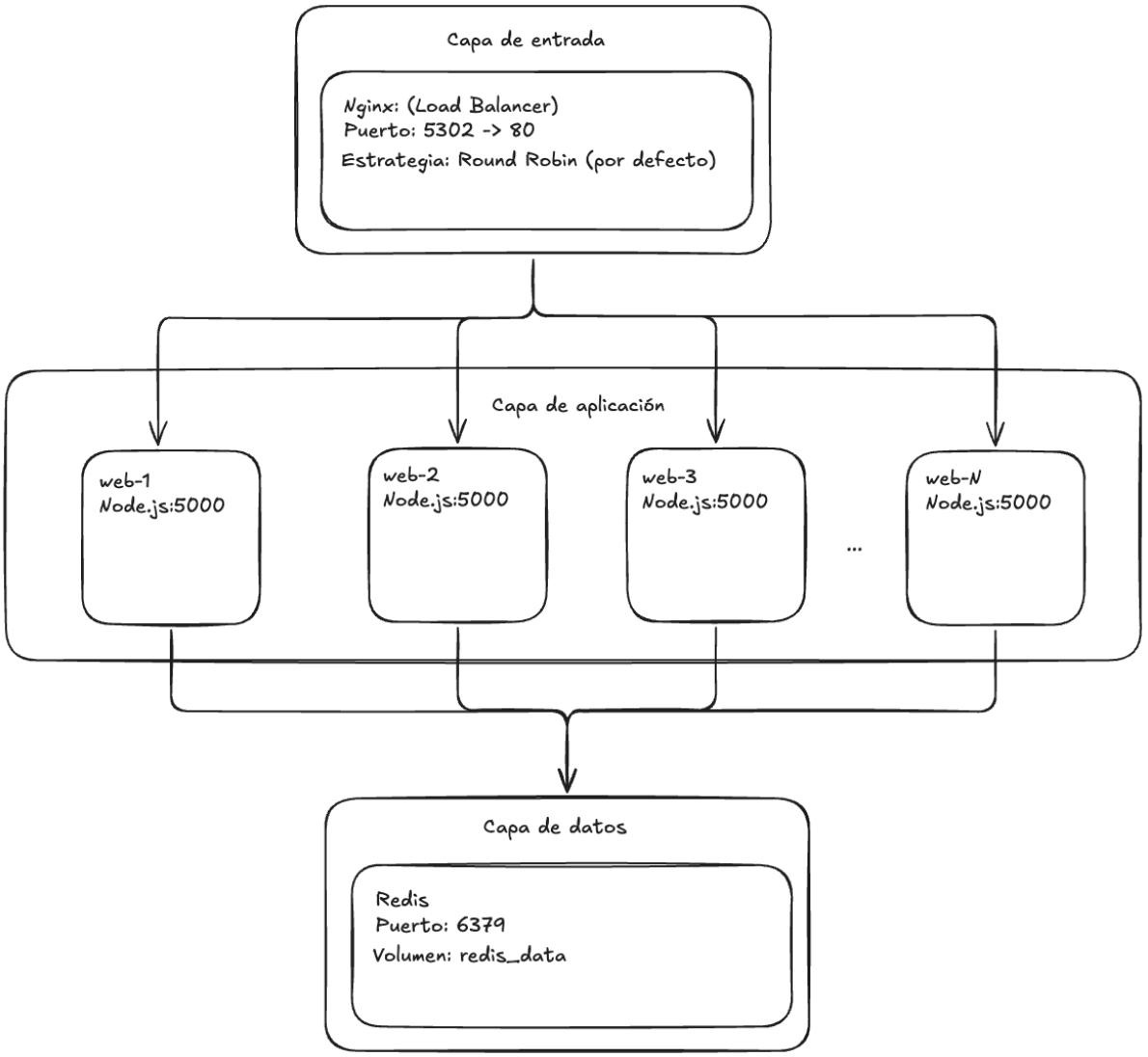
El primer paso consto de analizar un poco la aplicación en sí, comprender lo básico de como levantar una aplicación de node.js y dejarla funcionando en los ambientes locales.
De este punto, descubrimos que era necesario separar las cosas en dos dockerfile, uno para los contenedores de la aplicación y otro para el nginx.
A partir de ahí todo fue fluyendo, nos dimos cuenta que si usábamos dos dockerfiles distintos, sería pertinente tener contenedores separados tambien, ahi llegamos a la conclusión que debíamos realizar 3 contenedores distintos, uno con nginx, otro con la aplicación y otro con redis.
Para cada uno de los contenedores fuimos mirando la letra y analizamos cada una de sus necesidades.

1. Empaquetado de la aplicación web:
    a. Tanto el dockerfile como la configuración correcta de dependencias quedaron resueltas al levantar un contenedor para utilizarla.
    b. Revisamos en qué puerto se servía la aplicación y seteamos el puerto de comunicación de la imagen en el 5000.
2. Configurar la persistencia de datos mediante volúmenes:
    a. Lo primero que hicimos fue investigar un poco que era redis y como funcionaba, de ahí llegamos a un foro de stackoverflow donde alguien explicaba como funcionaba un volumen con redis, ya que necesitábamos que la información persistiera. Usando lo que explicaba generamos la persistencia.
    b. Para garantizar la comunicación de la aplicación con redis y de la aplicación con nginx agregamos todos los contenedores a una red custom que funciona en modo puente, permitiendo que los contenedores se comuniquen entre sí.
3. Implementación del balanceador de carga:
    a. Para definir el balanceador de carga, utilizamos el bloque http junto con el bloque upstream, con esto definimos un grupo de servidores llamado “loadbalancer”.
    Docker compose se encarga de resolver webapp.
    b. Luego, utilizamos el bloque server, establecimos el puerto 80 para la comunicación dentro del contenedor y expusimos el puerto 5302 fuera del docker, permitiendo la comunicación externa solo con nginx y que el resto de servicios se resuelvan internamente.
    c. Agregamos un server_name para definir a donde debíamos enviar las peticiones en local, en un entorno de producción debe setearse un dominio real.
    d. Por último, utilizamos el bloque location, clave para redirigir la petición al grupo loadbalancer definido anteriormente, con esto lo dejamos

configurado para que reciba la petición, la reenvíe y devuelva la respuesta.

Dentro de los otros requerimientos, configuramos dependencias lineales entre contenedores, la aplicación web depende de redis y nginx depende de la aplicación web.

4. Diagramas de comunicación



5. Análisis de ventajas y desventajas de la solución implementada

5.1 Ventajas

Ventajas	Descripción
Alta disponibilidad	Con el balanceador se genera una disponibilidad extremadamente alta, permitiendo que si fallara una de las instancias se pueda enviar el tráfico de la misma otra.
Escalabilidad horizontal	Permite ajustar la capacidad de recursos de manera práctica y simple, solo restaría agregar más valores a la cantidad de instancias requeridas. Ideal para cloud.
Distribución de carga	Mejora el rendimiento, tiempos de respuesta más rápidos y evitamos que alguno de los servidores se sature.
Despliegue sin downtime	A la hora de hacer despliegues, nos encontramos con la ventaja de que podemos actualizar una sola de las instancias, permitiendo verificar funcionamiento y posteriormente actualizar el resto de las instancias.

6. Desventajas

Desventajas	Descripción
Mayor costo técnico de mantenimiento	Se complejiza la estructura de mantenimiento (dos dockerfiles, un docker compose y nginx).
Consumo de recursos	Se aumenta el uso de recursos (cuatro instancias consumen más que una sola).
Gestión de estado compartido	Dos instancias no pueden modificar lo mismo al mismo tiempo. Es necesario una sincronización para mantener consistencia y fiabilidad en los datos.

Desventajas	Descripción
Debugging complejo	Es necesario revisar múltiples logs e incluso dificultar su lectura si no están bien trabajados.

6. Propuestas de mejoras para entorno de producción

Para una implementación en un entorno de producción real, se proponen las siguientes mejoras:

Mejora	Descripción
Logging centralizado	Actualmente los logs se encuentran dispersos en cada contenedor, se podría usar algún concentrador de logs (loki + grafana).
Monitorización	Se podría implementar un sistema de monitorización para conocer estadísticas de la aplicación (requests por hora, uso de CPU/RAM por contenedor, etc).
Respaldo automático de la base de datos	Generar respaldos automáticos de la base de datos, en caso de fallas, poder recuperar el contenido.